Jacob Luke
Course: ECE 601
Instructor: Marco Duarte

Project Phase 2: Part (d)

**Model Architecture:**

**Platform:** The platform used for this model was Pytorch in a Jupyter notebook run in a Google Colab environment.

**Input Data:**

- 50,000 Training Samples
- 10,000 Test Samples
- Each sample is a 32 x 32 RGB image
- No data augmentation

**Convolutional Layers:** The generated model use 3 blocks of convolutional layers following the pattern;

Conv2d — BatchNorm2d — ReLU — Conv2d — BatchNorm2d — ReLU — MaxPool2d

All together the convolutional layers look like this;

Block 1: (Conv2d — BatchNorm2d — ReLU — Conv2d — BatchNorm2d — ReLU — MaxPool2d)

Block 2: (Conv2d — BatchNorm2d — ReLU — Conv2d — BatchNorm2d — ReLU — MaxPool2d)

Block 3: (Conv2d — BatchNorm2d — ReLU — Conv2d — BatchNorm2d — ReLU — MaxPool2d)

Dimensionality is only changed;

- For channels, at the first Conv2d layer of each block. This is because Conv2d layers are all configured with stride = (1, 1), padding = (1, 1) and kernel = (3, 3) preserving spatial dimensions along Height and Width.
- For spatial dimensions, at the MaxPool layer that ends each block. MaxPools are oriented with stride = 2 and kernel_size = 2, this setup halves the spatial dimensions each time MaxPool is called.

For the 32 x 32 RGB (3 channels) images that are fed into the model the dimensionality reduction flow looks like;

- Input: 3 x 32 x 32 (C x H x W)
- Block 1: The first Conv2d outputs 64 x 32 x 32. MaxPool outputs 64 x 16 x 16.
- Block 2: The first Conv2d outputs 128 x 16 x 16. Maxpool outputs 128 x 8 x 8.
- Block 3: The first Conv2d outputs 256 x 8 x 8. Maxpool outputs 256 x 4 x 4.
- Output: 256 x 4 x 4

Other Notes: The generated model has bias=False for all Conv2d layers. The model also has a safety net pooling layer (AdaptiveAvgPool2d) which ensures that the output of the convolutional layers output is of the spatial dimensions 4x4 before it is flattened. The data is then flattened to be fed into the Fully Connected Layers, the dimension of the flattened data is 256 x 4 x 4 = 4096 data points.

**Fully Connected (FC) Layers:** The fully connected section of the model is multi-head. Since the task is a multi label classification the fully connected section has two heads one to classify the images to their "coarse" (20 classes) label and one to classify images to their "fine" (100 classes) label. Each of these heads has the same architecture essentially only deviating in how they reduce dimensions. Each head follows the architecture;

Linear — ReLU — Dropout — Linear

For the head handling the "fine" labels the dimensionality reduction looks like;
- Input 4096 features
- Linear layer 1 outputs 512 features.
- Linear layer 2 outputs 100 features.

For the head handling the "coarse" labels the dimensionality reduction looks like;
- Input 4096 features
- Linear layer 1 outputs 256 features.
- Linear layer 2 outputs 20 features.

As noted above the model uses dropout. The dropout layer in both heads has p=.4 meaning it drops 40% of the neurons when training.

Overall there are a total of 4,349,240 trainable parameters.

**Optimization:** The AdamW optimizer is called. This differs from the purely Adam optimizer because instead of adding weight decay at the beginning of the optimization computation it waits

until right before the weight is updated to apply the weight decay. This is done to avoid injecting noise into gradients that Adam uses to calculate its moving averages.

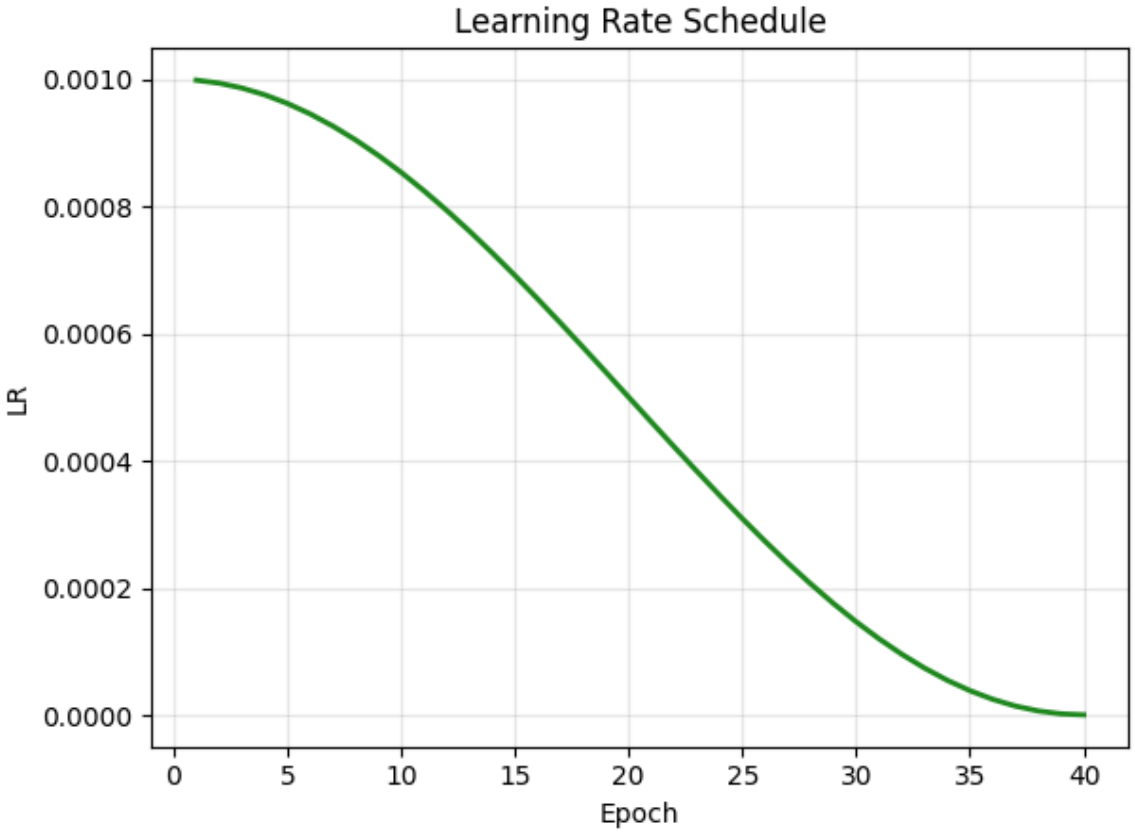
The learning rate of the model is a scheduled learning rate used to decrease the learning rate as training progresses to ensure convergence. The specific scheduler is called the CosineAnnealingLR. This specific learning rate schedule updates the learning rate before every new epoch and is calculated as;

$$\eta_{new} = \eta_{minimum} + \frac{1}{2}(\eta_{current} - \eta_{minimum})\left(\frac{1 + cos\left(\frac{(T_{current} + 1)\pi}{T_{maximum}}\right)}{1 + cos\left(\frac{T_{current}\pi}{T_{maximum}}\right)}\right) \quad [1]$$

The given model has,

$\eta_{minimum}$ = 1e-6 (Final learning Rate)

$T_{maximum}$ = 40 (Total number of epochs)




Pytorch's official documentation for this scheduler can be found at [1].

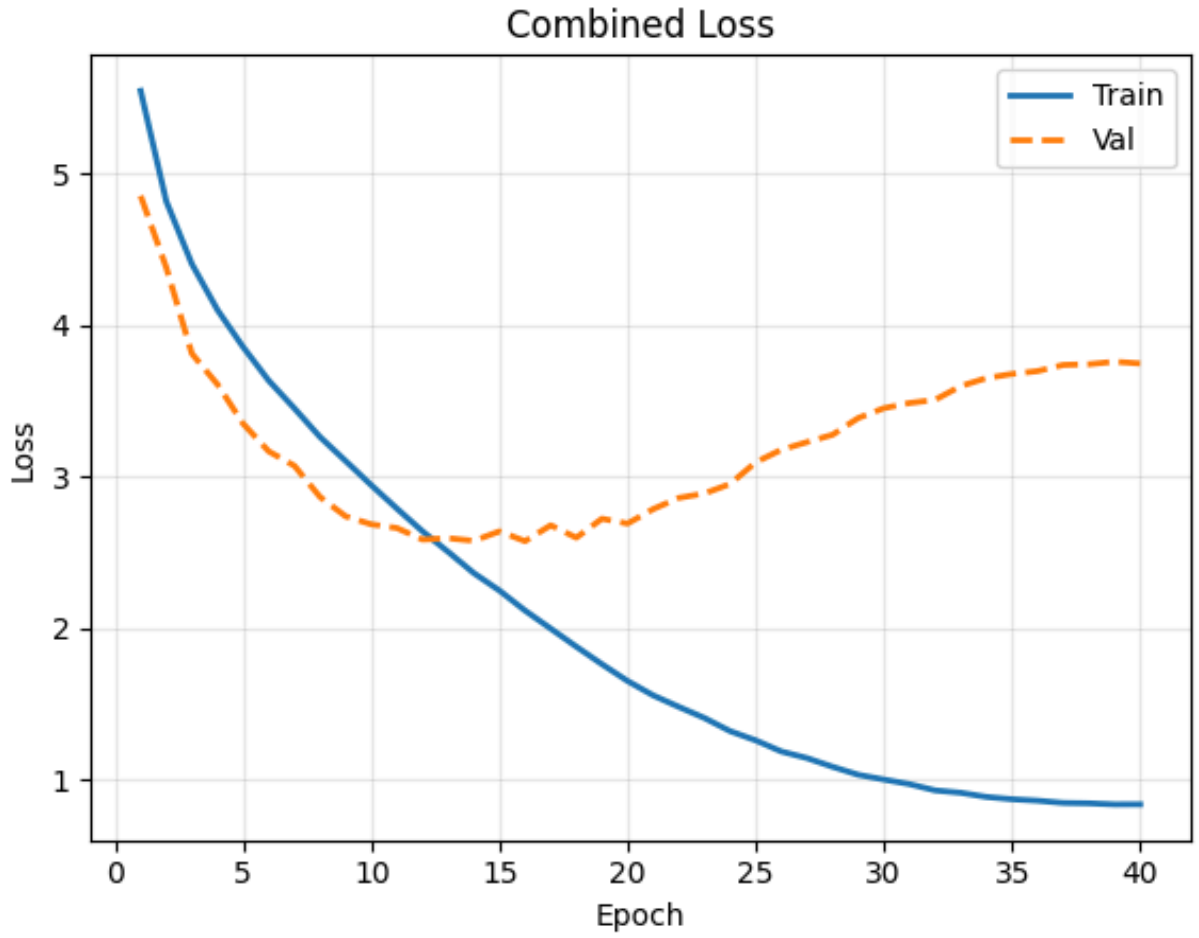
**Batch Training:** The model was trained in batches of 128 totaling 391 batches per an epoch.

**Cost Function:** The cost function of the given model is Cross Entropy Loss.

**Other Notes:** The generated model was trained on 50,000 training samples but shows during training that there is a validation test statistic. I am noting this because the generative LLM decided to get validation loss by evaluating the entire test set after every epoch of training instead of holding out a percentage of the training set for validation. This is hugely problematic because it then used the model that performed best on the "validation" set to save as the best model. It quite literally cherry-picked the model that did best on the test set, breaking the possibility of a useful performance evaluation using unseen data. It is also hugely problematic for tuning hyperparams in the future because I would essentially be tuning the model to perform better on the test set directly if I was not aware of this.

**Results:**

**Training Set:** During training it is clear that the model starts to overfit around epoch 13.




The divergence of the validation loss is a clear indication that the model is overfitting to the training data.

**Test Set:**
Fine Top-1 Accuracy: 50.32%
Fine Top-5 Accuracy: 79.31%
Coarse Top-1 Accuracy: 67.75%

These test set statistics tell us that the model is quite inadequate for the “fine” labels which is the 100 class classification task. However, it does give us the insight that the model often had the correct label in the top 5 but then made the wrong prediction out of those top 5 options. The model also performed quite poorly on the “coarse” labels as well settling at 67.75% accuracy on the 20 class classification task. For comparison in [2] it is reported that they were able to achieve 89.74% Top-1 accuracy for the 100 class classification task on this dataset using a vision transformer with 185,956 trainable parameters. That means they achieved ~40% higher accuracy with a model that has 95.7% less trainable parameters than the one generated. Of course that is a different architecture, however, in the GitHub repository of models trained on cifar-100 in [3] you can find a network named “mobilenet” which is a CNN deployed using Pytorch with 3.3 million trainable parameters where they reported 66% test accuracy which is still about 16% more accurate than the generated model with a similar number of parameters indicating that the model has much room for improvement.

References:

[1] "CosineAnnealingLR," PyTorch 2.11 documentation. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html. [Accessed: Apr. 2026].

[2] P. Mondal, "vit-lora-cifar100," Hugging Face, Oct. 2023. [Online]. Available: https://huggingface.co/priyadip/vit-lora-cifar100. [Accessed: Apr. 2026].

[3] Weiaicunzai, "pytorch-cifar100," GitHub, 2022. [Online]. Available: https://github.com/weiaicunzai/pytorch-cifar100. [Accessed: Apr. 2026].